

Customer Churn Data Preparation

Methods and results for cleaning, splitting, encoding, scaling and validation

Item	Details
Project	Customer Churn Analysis
Deliverable	Data Preparation
Role	Data Engineer
Source dataset	7,043 records and 10 variables
Prepared output	Training and testing data ready for further analysis

Data Preparation Overview

The source data contained 7,043 customer records and 10 variables, with Churn used as the target variable. The final outputs include a cleaned dataset, separate training and testing files, a fitted preprocessing pipeline, and validation reports.

Variable group	Variables	Preparation method
Categorical predictors	gender, Dependents, PhoneService, MultipleLines, InternetService, Contract	One hot encoding
Continuous predictors	tenure, MonthlyCharges	StandardScaler
Binary predictor	SeniorCitizen	Passed through without scaling
Target	Churn	No = 0 and Yes = 1

Workflow

- The source data was inspected and checked for quality issues.
- Confirmed logical conflicts were corrected, and each decision was documented.
- Churn was separated from the predictors before an 80 to 20 stratified split was performed.
- The encoder and scaler were fitted on the training data only.
- Both datasets were transformed and validated.
- The datasets, reports, and fitted pipeline were saved for later use.

Data Integrity and Cleaning

Before any changes were made, the data was checked for missing values, blank strings, duplicate rows, invalid categories, impossible numerical ranges, and IQR based outliers. Related fields were also compared to identify inconsistent combinations.

Check	Finding	Decision
Missing values	0	No action required
Blank strings	0	No action required
Fully duplicated rows	302	Retained because no unique customer ID was available
Invalid categorical values	0	No action required
Impossible numerical values	0	No action required
IQR based outliers	0	No action required
Phone service logical conflicts	260	Changed MultipleLines to No

Duplicate record decision

A total of 302 rows had identical values. These rows were retained because the dataset did not include a unique customer ID. They may represent different customers with the same details, so removing them could delete valid records.

Logical consistency correction

The data contained 260 records marked as having multiple lines even though PhoneService was No. Because this combination is not logically possible, MultipleLines was changed to No for those records. The rule was checked again after correction. No conflicts remained, and the dataset still contained 7,043 rows.

```
logical_conflict_mask = (  
    (df_clean["PhoneService"] == "No") &  
    (df_clean["MultipleLines"] == "Yes")  
)  
  
df_clean.loc[logical_conflict_mask, "MultipleLines"] = "No"
```

Training and Testing Split

The cleaned data was split before encoding or scaling. Churn was mapped separately, using 0 for No and 1 for Yes. This order kept the testing data out of the fitting process.

Dataset	Rows	Non churn	Churn	Churn rate
Full dataset	7,043	5,174	1,869	26.54%
Training set	5,634	4,139	1,495	26.54%

Dataset	Rows	Non churn	Churn	Churn rate
Testing set	1,409	1,035	374	26.54%

Why an 80 to 20 split was used

Eighty percent of the records were assigned to training, while 20 percent, or 1,409 records, were reserved for testing. Stratification by Churn preserved the original 26.54 percent churn rate in both sets. A random_state of 42 was used so the split can be reproduced.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

Categorical Encoding

Six predictor columns contained text categories. They were converted into binary columns with OneHotEncoder so they could be used in numerical analysis.

- drop='first' removes one reference category from each feature and avoids redundant dummy variables.
- handle_unknown='ignore' allows the fitted pipeline to process a previously unseen category without failing.
- sparse_output=False returns a dense array that can be converted directly into a labelled pandas DataFrame.

Original feature	Encoded output
gender	gender_Male
Dependents	Dependents_Yes
PhoneService	PhoneService_Yes
MultipleLines	MultipleLines_Yes
InternetService	InternetService_Fiber optic
Contract	Contract_One year and Contract_Two year

Feature Scaling

The tenure and MonthlyCharges variables were standardised with StandardScaler. For each value, the scaler subtracts the training mean and divides the result by the training standard deviation.

$$z = (x - \text{training mean}) / \text{training standard deviation}$$

Why StandardScaler was selected

tenure is measured in months, while MonthlyCharges is a monetary value. Their ranges are different. Standardising them puts the two features on a comparable scale and stops the larger range from dominating distance calculations.

Features included and excluded

Feature	Treatment	Reason
tenure	StandardScaler	Continuous value measured in months
MonthlyCharges	StandardScaler	Continuous monetary value with a different range
SeniorCitizen	Passthrough	Already represented as a binary 0 or 1 value
One hot encoded columns	No scaling	Already represented as binary 0 or 1 values

Scaling code

```
preprocessor = ColumnTransformer(  
    transformers=[  
        ("categorical", OneHotEncoder(  
            drop="first",  
            handle_unknown="ignore",  
            sparse_output=False  
        ), categorical_features),  
        ("continuous", StandardScaler(), continuous_features),  
        ("binary", "passthrough", binary_numeric_features)  
    ],  
    remainder="drop",  
    verbose_feature_names_out=False  
)
```

Result of scaling

After scaling, tenure and MonthlyCharges both had a training mean of 0.0 and a population standard deviation of 1.0. This matched the expected result.

Data Leakage Prevention

The preprocessing pipeline was fitted on X_train only. The same fitted pipeline was then used to transform X_test. As a result, the test categories, means, and standard deviations did not affect the training process.

```
X_train_processed_array = preprocessing_pipeline.fit_transform(X_train)  
X_test_processed_array = preprocessing_pipeline.transform(X_test)
```

If the scaler had seen the full dataset before the split, the test data would have influenced the transformation. Keeping the test set separate gives a more reliable evaluation.

Processed Dataset Structure

The processed training file has 5,634 rows and 10 feature columns. The testing file has 1,409 rows with the same columns. Target values were saved in separate files, while the original record indices were retained so that each X row still matches the correct y value.

Position	Processed feature	Type
1	gender_Male	Encoded binary
2	Dependents_Yes	Encoded binary
3	PhoneService_Yes	Encoded binary
4	MultipleLines_Yes	Encoded binary
5	InternetService_Fiber optic	Encoded binary
6	Contract_One year	Encoded binary
7	Contract_Two year	Encoded binary
8	tenure	Standardised continuous
9	MonthlyCharges	Standardised continuous
10	SeniorCitizen	Existing binary

Post Transformation Validation

Thirteen checks were completed after transformation, and all of them passed. The results confirmed that no rows were lost, both datasets used the same columns, and no missing or infinite values were introduced.

Validation area	Result
Training and testing row counts	Preserved
Training and testing feature columns	Matched
Missing values	0 in both datasets
Infinite values	0 in both datasets
Encoded feature values	Binary in both datasets
Target values	Binary
Training and testing index overlap	0
Scaled training means	Approximately 0
Scaled training standard deviations	Approximately 1